

GIST: Goals as Implicit Supervision at Training for Visuomotor Policy Learning

Abstract— We present GIST, an end-to-end visuomotor policy trained with an auxiliary sub-goal prediction loss for manipulation tasks with non-prehensile components. Adding structure to behaviour cloning manipulation policies has been shown to improve robustness and generalization, with sub-goal based hierarchies predicting intermediate sub-goals with a high-level model and leaving a low-level policy to reach them. These sub-goals are typically extracted at gripper open-close frames, and hence the hierarchy is learned around this discrete signal. Non-prehensile settings, such as pouring or sweeping, hold the gripper in a fixed configuration, so no such open-close events occur and transitions go undetected. GIST instead uses sub-goals purely as a training signal. An auxiliary head reads the visual tokens the policy acts on and predicts a weighted Gaussian mixture over the next sub-goal gripper pose, placing a mode on each plausible sub-goal instead of collapsing to an average. The head is discarded after training. Across four simulation and three real-world tasks, GIST outperforms a standard sub-goal based hierarchical policy by 15.2% and a stronger uncertainty-aware baseline that we formulate by 5.9%, while removing the auxiliary loss costs 10.7% in simulation and 33.3% in the real world. GIST also outperforms Diffusion Policy by 5.0% across five prehensile and non-prehensile MimicGen tasks, and by 4.4% in target area coverage on Push-T. Performance is largely insensitive to the sub-goal extraction procedure, indicating that what matters is the presence of spatial goal supervision, not its precise definition. Website: <https://gist-implicit-subgoal.github.io/>

I. INTRODUCTION

Robotic manipulation in household and industrial settings frequently involves sustained physical interaction rather than discrete grasping. Pouring from a container, closing a lid, sliding a container aside, and pushing an object into place are all non-prehensile interactions, in which the gripper presses or pushes an object without ever grasping it, holding a fixed configuration for the duration of the motion. Interactions of this kind appear throughout everyday environments, and are often the only way to manipulate an object that is too large, too flat, or too cluttered to grasp. We target tasks that contain such non-prehensile components, and learn them from demonstrations.

Behavior cloning maps observations directly to actions, with the demonstrated future action chunks as its only training signal [1], [2], [3], [4]. A common way to make such a policy more robust and more general is to give it structure, and prior work has done so in several forms, reducing demonstrations to sparse end-effector keyframes [5], [6], [7], predicting the motion of scene points [8], decomposing tasks into language sub-tasks [9], or learning latent plans from human play [10]. Closest to our setting, sub-goal based hierarchical policies predict an intermediate spatial target

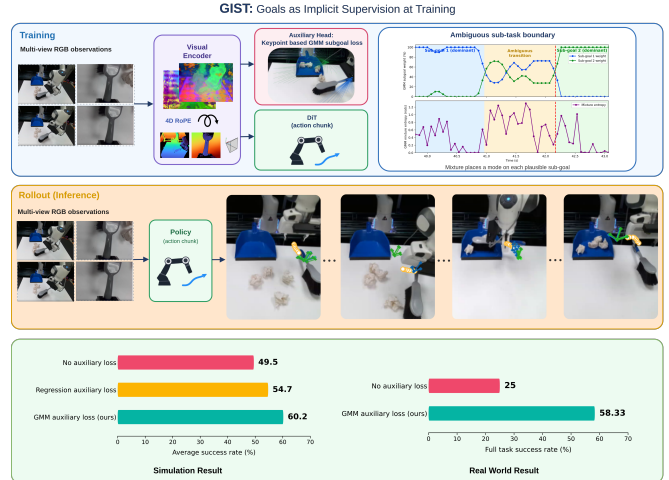


Fig. 1. **Top:** GIST predicts an action chunk from multi-view RGB observations, uses RoPE-4D to ground observation and uses a keypoint based GMM auxiliary loss during training, which is discarded during rollout. At an ambiguous sub-task boundary two sub-goals are plausible at once, shown inset in blue and green, and a mixture can place a mode on each where a single regressed pose cannot. **Bottom:** The bar charts report average success rate in simulation and full task success in the real world, against a policy trained without the auxiliary loss and one whose auxiliary head regresses a single sub-goal.

with a high-level model and leave a low-level policy to execute it [11], [12], [13]. The high-level model reasons about the semantic and visual content of the scene at a spatially coarser level, and produces the keypoints and sub-task targets that it sends to the low-level policy. The low-level policy is left with the control problem, generating action chunks that carry the gripper through the short-horizon motion towards the target it is given.

The structure that makes the sub-goal based hierarchical policies useful also makes it fragile, and in the non-prehensile tasks considered, it is fragile in two separate ways. The first concerns how sub-goals are extracted from the demonstration trajectories. To extract sub-goals, previous methods have used the gripper open-close signal to segment the demonstrations [11], [12]. When the gripper opens and closes at the meaningful transition points of a task, this segmentation is straightforward. In the tasks we target, however, the gripper holds a constant configuration across multiple such transition points, so these transitions go undetected, and the boundaries have to be recovered from the geometrical or physical properties of the trajectory instead. The second issue arises when training the high-level policy. On either side of a transition point the observations are nearly identical, and

with no gripper event the proprioception is identical as well, yet the target jumps from one sub-goal to the next. A high-level model trained to predict a single sub-goal therefore regresses towards the average of the present and the next sub-goal, and sends the low-level policy a pose that matches neither.

In contrast to sub-goal based hierarchical policies, we neither predict explicit sub-goals nor condition a low-level policy on them. We instead use the sub-goals purely as a training signal. During training, an auxiliary head reads the same visual tokens the policy acts on and predicts a weighted Gaussian mixture over the next sub-goal gripper pose, trained by minimising the negative log-likelihood of the demonstrated sub-goal, and this loss is added to the behaviour cloning objective under a single weighting coefficient. Once training ends the head is discarded, and the deployed policy runs on observation and proprioception alone. Similarly, in contrast to end-to-end policies, which learn from the demonstrated actions alone, the auxiliary sub-goal provides the policy with an explicit spatial signal about task progress during training, while leaving it unchanged at deployment.

Finally, because non-prehensile tasks have no explicit gripper open-close signal, we explore different methods to extract this keypoint. However, we found that improvement by using these different keypoint extraction strategies is not very significant for the auxiliary loss approach indicating that GIST is robust to learning the inductive biases from these goals irrespective of the goal decomposition method.

In summary, our contributions are as follows:

- We propose a new policy architecture that uses subgoals as *auxiliary supervision on the visual representation* instead of a sub-goal based hierarchical policy.
- We show that GIST outperforms the hierarchical formulation from prior work [11], [12] by **15.2%** and the *stronger uncertainty-aware hierarchical baseline that we formulate* by **5.9%**, while running no high-level model at inference.
- We isolate where the gain comes from. Removing the auxiliary head costs **10.7%** in simulation and **33.3%** in real-world success, and regressing a single subgoal instead of predicting the mixture costs **5.5%**. Both the supervision and its representation as a distribution matter.
- We systematically compare six sub-goal extraction strategies in the absence of a complete gripper open-close signal and find that GIST is largely insensitive to how the sub-goal boundaries are defined.
- We further compare against Diffusion Policy on five MimicGen tasks spanning *both* prehensile and non-prehensile manipulation, where GIST beats the strongest of its three variants by **5.0%** and the policy trained without the auxiliary loss by **8.4%**, and on Push-T, where it leads by **4.4%** in target area coverage.

II. RELATED WORKS

A. Learning for Non-Prehensile Tasks

Non-prehensile manipulation has a long history in model-based planning and control, from the mechanics of pushing and sliding [14], [15], [16], [17] to planning and control directly through contact [18], [19], [20], [21], [22]. A second family trains policies with reinforcement learning in simulation and transfers them to the real world [23], [24], [25], [26], [27], [28], and a third learns a dynamics model of how objects move under contact and acts by planning against it [29], [30], [31], [32]. Each relies on a resource we do not assume, whether an explicit contact model and accurate object state, privileged simulator state and reward design, or a learned model of object dynamics. We instead learn from a modest number of real demonstrations, and the deployed policy is a single reactive network.

Closest to our setting, non-prehensile behaviour appears in demonstration-based and generalist policies. Push-T is a standard benchmark for imitation methods [33], [1], Bi-NoMaP [34] learns bimanual non-prehensile primitives from human videos, and generalist policies [3], [9] exhibit pushing and pressing within larger tasks. None of them asks whether making intermediate goals explicit during training helps in this regime, or how such goals should be defined when no gripper event marks them. Our work poses that question, and answers it with an auxiliary goal-prediction objective that supplies the signal at training time without changing the policy’s inputs at inference.

B. Hierarchical and Non-Hierarchical Policy Learning

Non-hierarchical policies. End-to-end visuomotor policies map observations directly to actions, trained on the behaviour cloning objective alone [33], [1], [35], [2], and recent large-scale policies adopt the same formulation across many robots and tasks [36], [4], [37], [3], [38]. These works cover prehensile and non-prehensile behaviour alike, but none supervises an intermediate spatial target. Closer to us, auxiliary heads have been used to shape the representation the action head reads, with targets such as pixel and state reconstruction [39] or future latent embeddings [40]. Our auxiliary target is instead the next sub-goal gripper pose, a spatial signal about task progress that behaviour cloning alone does not provide.

Hierarchical policies. Hierarchical imitation learning factorizes the policy into a high-level model that proposes an intermediate target, often an end-effector pose or keyframe, and a low-level policy that realizes it [11], [12], [5], [6], [7]. The sub-goals that train these methods are segmented from the demonstrations, most often at gripper open-close events, which the non-prehensile portions of our tasks do not provide. We differ from these methods in two ways. The sub-goal is consumed entirely at training time, so the deployed policy is end-to-end and runs no high-level model, and GIST is robust enough to reason about the subgoals extracted through heuristics for tasks where gripper open-close does not extract sub-goals.

III. PROBLEM STATEMENT AND ASSUMPTIONS

In this work, a sub-goal is the gripper configuration at a frame where one stage of a demonstration ends and the next begins, and we write $g(t)$ for the next sub-goal at time t . We keep the term from hierarchical policy learning, but in our setting a sub-goal carries no semantic label and amounts to a *waypoint* of the end-effector trajectory, an intermediate pose the demonstration passes through. In grasp-based tasks the segmentation is canonical, since the gripper opens or closes at each transition and the segments correspond to nameable skills such as grasping and placing. We target tasks in which the transitions carry no such discrete event, so the boundaries are not directly observable.

At each timestep t the policy receives the T_o most recent observations and proprioceptive states, abbreviated o_t and s_t , and predicts a chunk of H actions $a_{t:t+H}$ (end index exclusive), of which the first H_{exec} are executed before replanning. We represent any gripper configuration by the 3D positions of $K=4$ keypoints. We write $e_t \in \mathbb{R}^{K \times 3}$ for the current gripper, computed from s_t so that proprioception is grounded in the frame of the visual observations, and the sub-goal defined above is likewise a set of K keypoints, so $g(t) \in \mathbb{R}^{K \times 3}$.

Data and assumptions. We assume demonstrations $\mathcal{D}_{\text{robot}} = \{(o_t, s_t, a_t)\}$, each a successful execution of a single task, with no language conditioning or task identifier. Here o_t is calibrated multi-view RGB and metric depth with known intrinsics and extrinsics, s_t is the proprioceptive state vector, and a_t is the recorded action. The sub-goals $g(t)$ are extracted offline from the end-effector trajectory, without privileged access to object state or contact events. Since no gripper event marks the boundaries, we compare a family of procedures operating on the trajectory alone, from trajectory simplification and velocity changepoints to uninformative controls, which differ only in where they place the boundaries and are studied in Section V-B. Hardware and per-task data collection details are in Appendices IV and V of the supplementary material.

IV. METHOD

A. OVERVIEW

We describe GIST, denoted π_{aux} , shown in Figure 2, a single end-to-end network trained with two objectives. The first is the flow-matching behaviour cloning objective on the demonstrated action chunks. The second is an auxiliary objective in which a head reads the same grounded visual tokens the action head attends to and predicts a distribution over the next sub-goal $g(t)$. The auxiliary head is discarded once training ends, so the sub-goals shape the visual representation during training without appearing in the policy’s input at deployment. We first describe how the visual observations are grounded in 3D, then the auxiliary goal head, then the action generation, and finally how the two objectives are combined.

B. POLICY LEARNING

We instantiate π_{aux} as a flow-based DiT policy [41], [42], [38] that generates an action chunk $a_{t:t+H}$ conditioned on the observation o_t . We use the keypoints (subgoals) $g(t)$ to supervise an auxiliary goal-prediction loss that shapes the visual representation the action head attends to.

a) 3D-Grounded Visual Tokens

A frozen DINOv2 backbone [43] encodes each of the C camera views at each of the T_o observation steps into N_p patch tokens, projected to the model width with learned camera and observation-step embeddings. Unprojecting each patch centre with the depth image and the camera calibration assigns it a world-frame anchor $\mathbf{x}_{c,i} \in \mathbb{R}^3$, with patches lacking valid depth masked out ($v_{c,i} = 0$). Following ArticBot [11], the current gripper keypoints e_t are encoded by an MLP into K further tokens, anchored at their own positions.

The patch and gripper tokens form a single sequence, processed by $L=2$ pre-norm transformer blocks whose self-attention uses a four-dimensional (4D) rotary position embedding (RoPE) [44] over $(\lambda \mathbf{x}, \tau)$, with λ a metric scale and $\tau = k/(T_o + H)$ placing observation step k on a time axis whose value 1 marks the end of the action horizon. Rotating only the queries and keys lets relative 3D geometry decide which tokens attend to one another, without writing position into token content. We write $\tilde{z}(o_t)$ for the resulting grounded patch tokens, which serve as the cross-attention context of the DiT and, with the gripper tokens and their anchors, as the input to the auxiliary head.

b) Auxiliary Goal Mixture Head

Following ArticBot [11] and GHOST [12], we parameterise $g(t)$ as a dense per-anchor Gaussian mixture. A shared MLP maps each grounded token concatenated with its world-frame position, $[\tilde{z}_n; \mathbf{x}_n]$, to a displacement $\Delta_n \in \mathbb{R}^{K \times 3}$ and a logit ℓ_n , giving a component $\mu_n = \mathbf{x}_n + \Delta_n$ with softmax weight w_n . Unlike [12], which uses one component per image patch, the K gripper tokens are anchors too, giving $N_a = K + CN_p$ components. Writing $\mathcal{N}_n^\sigma = \mathcal{N}(g(t); \mu_n, \sigma^2 I)$ and summing over the N_v anchors with valid depth, the head is trained by negative log-likelihood over a variance ladder Σ , each rung paired with a uniform-weight term that keeps every anchor regressing a plausible offset,

$$\mathcal{L}_{\text{goal}} = - \sum_{\sigma^2 \in \Sigma} \left[\log \sum_{n:v_n=1} w_n \mathcal{N}_n^\sigma + \lambda_u \log \sum_{n:v_n=1} \frac{1}{N_v} \mathcal{N}_n^\sigma \right]. \quad (1)$$

The head is discarded at deployment.

c) Action Generation

The DiT operates on a short stream of T_o proprioceptive state tokens followed by H noised action tokens, alternating self-attention within the stream with cross-attention to the grounded visual tokens $\tilde{z}(o_t)$, with adaptive layer normalisation conditioned on the flow time. Given a normalised action chunk $a_{t:t+H} \sim \mathcal{D}_{\text{robot}}$, noise $\epsilon \sim \mathcal{N}(0, I)$ and flow time $\alpha \in$

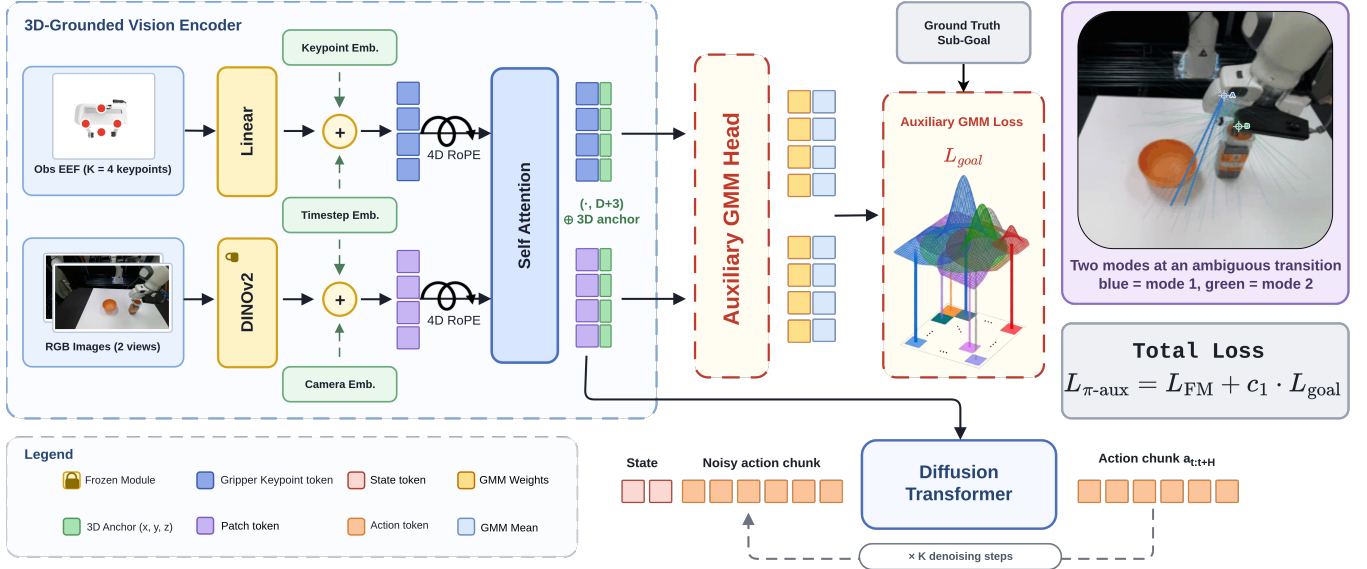


Fig. 2. Architecture of GIST. A frozen DINOv2 backbone and a self-attention trunk produce visual tokens anchored at their unprojected 3D positions. The auxiliary head reads these tokens and predicts a Gaussian mixture over the next sub-goal, placing a mode on each plausible sub-goal at an ambiguous boundary, shown on the right in blue and green. Its loss reaches only the head and the shared encoder, never the action head, so the sub-goals shape GIST purely through the visual representation. The auxiliary head is discarded at deployment.

$[0, 1]$, we form the interpolant $a_{t:t+H}^\alpha = (1 - \alpha) \epsilon + \alpha a_{t:t+H}$ and train the model to predict its velocity $u = a_{t:t+H} - \epsilon$,

$$\mathcal{L}_{FM} = \mathbb{E}[\|u - v_\theta(a_{t:t+H}^\alpha, \alpha, \tilde{z}(o_t), s_t)\|^2], \quad (2)$$

with α drawn from the Beta-based schedule of [38].

d) Training

We train on $\mathcal{D}_{\text{robot}}$ with $\mathcal{L}_{\pi_{aux}} = \mathcal{L}_{FM} + c_1 \mathcal{L}_{goal}$ ($c_1=0.1$), where $g(t)$ is the gripper keypoints at the next sub-task boundary of the demonstration (Sec. V-B). All parameters except the frozen DINOv2 are trained from scratch. The goal head receives gradient only from $\mathcal{L}_{goal}$ and the DiT only from $\mathcal{L}_{FM}$, while both losses reach the grounded encoder and hence the goal influences π_{aux} only by shaping the visual tokens. We set c_1 so that the auxiliary gradient at the encoder is $\sim 25\%$ of the flow-matching gradient at initialisation (hyperparameters in Appendix III of the supplementary material).

e) Inference

At deployment the goal head is discarded: π_{aux} runs the grounded encoder once and the DiT on the observation alone, generating an action chunk $a_{t:t+H}$ and executing the first H_{exec} actions.

V. EXPERIMENTS AND RESULTS

We evaluate GIST on tasks where the gripper open close signals do not carry the entire meaning, or in other words, the task partly or fully is non-prehensile. Through our experiments we try to answer the following questions:

- Q1: Does having explicit sub-goal reasoning help the policy reason about non-prehensile tasks?
- Q2: Does a simple end-to-end policy using sub goals as auxiliary loss perform equivalent to or better than hierarchical policies?
- Q3: Do the keypoint decomposition techniques have any influence on the performance of the policy?

A. Tasks and Evaluation Protocols

We evaluate on four simulation tasks and three real-world tasks.

Simulation Tasks. We use Coffee Preparation D1, Hammer Cleanup D1 and Kitchen D1 from MimicGen [45], trained on 100 demonstrations each, and Push-T [1], trained on 206 demonstrations. We save a checkpoint every 20 epochs and roll each out for 150 episodes, three blocks of 50 with distinct base seeds, where each episode’s own seed fixes the initial scene configuration, so no two episodes repeat. An episode ends on success or at a fixed per-task step limit. Success is the environment’s task check for the MimicGen tasks, and for Push-T we define it as the T-shaped block covering more than 0.95 of the target region at some point in the episode. We report the success rate of the best checkpoint across epochs, with the episode budget, seeds and step limit identical for every method on a given task.

Real-World Tasks. The real-world tasks are coffee-bean pouring, sweeping and pushing a block to a target position. Each policy is trained on 100 teleoperated demonstrations and evaluated on 20 distinct initial scene configurations, in which the placement of every object in the environment is varied so that the configurations span the entire workspace. We report two metrics: Full task success counts a rollout as successful only if every subtask is completed. Partial task success measures how far a rollout progresses. For each rollout we record the percentage of consecutive subtasks completed from the start of the task, and report the average over rollouts. A rollout that completes every subtask up to and including the last is counted as a full success.

B. Data Collection and Keypoint Extraction

Data Collection. For the MimicGen tasks we use the demonstrations generated by MimicGen from its source

TABLE I

COMPARISON WITH BASELINES ON THE SIMULATION TASKS. WE REPORT SUCCESS RATE (%), WITH “AVG.” THE AVERAGE OVER THE FOUR SIMULATION TASKS. EVERY METHOD THAT USES SUB-GOALS USES THE SAME AWE GREEDY EXTRACTION WITH THE GRIPPER SIGNAL AT A THRESHOLD OF 0.35. BEST PER COLUMN IN BOLD.

Method	Kitchen D1	Coffee Prep. D1	Hammer Cleanup D1	Push-T	Avg.
No auxiliary loss (Q1)	49.3	43.3	47.3	58.0	49.5
Regression auxiliary loss (Q1)	58.7	50.0	54.0	56.0	54.7
Standard hierarchical (Q2)	54.0	38.7	47.3	40.0	45.0
Uncertainty-aware hierarchical (Q2)	58.0	52.0	49.0	58.0	54.3
GIST (ours)	60.7	68.0	50.0	62.0	60.2

TABLE II

REAL-WORLD RESULTS. FULL TASK SUCCESS (%) AND PARTIAL TASK SUCCESS (%), THE PERCENTAGE OF CONSECUTIVE SUB-TASKS COMPLETED, AVERAGED OVER ROLLOUTS. AVERAGES ARE OVER THE THREE TASKS, EXCEPT THE PARTIAL AVERAGE, WHICH IS OVER THE TWO TASKS THAT REPORT ONE. BEST PER COLUMN IN BOLD.

Method	Pushing		Sweeping		Pouring		Avg.	
	Full	Part.	Full	Part.	Full	Part.	Full	Part.
No auxiliary loss	35	N.A.	30.0	46	10.0	35	25.0	40.5
GIST (ours)	65	N.A.	70.0	70	40.0	52.5	58.3	61.3

demonstrations, and for Push-T we use the demonstrations released with Diffusion Policy. Each real-world demonstration is collected by teleoperating the robot through a complete execution of the task, and we record the multi-view RGB and depth images, the camera calibration, the proprioceptive state and the commanded actions at every step. The number of demonstrations per task is given in the previous subsection.

Sub-goal Extraction. In our tasks, the gripper signal marks only a fraction of the sub-goal transitions, so the remaining boundaries must be extracted from the demonstration itself. We compare six procedures:

- **AWE-Greedy** and **AWE-DP**: The greedy and dynamic-programming variants of AWE [46], which select waypoints that best reconstruct the trajectory.
- **Heuristic**: A combination of heuristics based on the gripper state and end-effector orientation (see supplementary material for details).
- **Fixed interval**: A boundary every K steps.
- **Bayesian Velocity Changepoints**: Bayesian online changepoint detection applied to the end-effector speed (Appendix II of the supplementary material).
- **VLM**: Qwen3.6, from the Qwen3 family [47], prompted with annotated contact sheets of subsampled frames and a short task description, identifies the frames at which discrete manipulation events are completed. The prompt is task-agnostic apart from the task description (Appendix VI of the supplementary material).

For methods that operate on the trajectory, observed gripper events are retained as boundaries alongside the extracted ones. All six procedures run on the same demonstrations and differ only in where they place the boundaries, isolating their effect on π_{aux} in Section V-E.

C. Baselines

We compare GIST against four baselines, each chosen to answer one of the questions above, on the four simulation tasks of Table I. These four are trained on the same demonstrations and, wherever a sub-goal is used, on the same sub-goals $g(t)$. We additionally compare against different variants of Diffusion Policy [1] on a separate set of five prehensile and non-prehensile MimicGen tasks and on Push-T, as shown in Table IV.

1) No Auxiliary Loss (Q1)

This baseline is π_{aux} with the auxiliary head and its loss removed, trained with $\mathcal{L}_{FM}$ alone. Every other component is identical, as are the seed, the data order, the batch size and the number of epochs, and because the goal head is initialized last, the two models start from the same shared weights and see the same batches. The auxiliary gradient is therefore the only difference, which isolates the effect of the sub-goal supervision itself (Q1).

2) Regression Auxiliary Loss (Q1)

Here the Gaussian-mixture loss on the token outputs of the grounded visual encoder is replaced by a regression loss, in which the head predicts a single sub-goal as the K gripper keypoints relative to the current gripper and is trained with mean squared error. Everything else is unchanged, and the loss weight is set so that the auxiliary gradient at the encoder matches that of π_{aux} at initialisation. Since a regression head predicts one sub-goal per observation, at an ambiguous boundary it is trained towards the average of the present and the next sub-goal. This separates the value of sub-goal supervision from the value of representing it as a distribution (Q1).

3) Standard Hierarchical Policy (Q2)

Following ArticBot [11] and GHOST [12], the high-level policy commits to one sub-goal and the low-level policy is conditioned on it. The high-level policy is the same π_{hi}

used by the uncertainty-aware hierarchical baseline described next, except that we keep only its highest-weight anchor, collapsing the mixture to a single pose. The low-level policy encodes that pose with an MLP into one goal token per observation step, alongside the state and noised action tokens in the DiT, and takes its visual tokens from the same frozen DINOv2 backbone without 3D anchors or the RoPE trunk, matching prior hierarchical low-level policies. It is trained on the ground-truth $g(t)$ and sees predicted sub-goals only at deployment. This tests whether committing to a single sub-goal can handle ambiguous boundaries (Q2).

4) *Uncertainty-Aware Hierarchical Policy (Q2)*

This is a stronger baseline that we formulate ourselves. Rather than collapsing the high-level output to a single sub-goal, it passes the entire distribution predicted by the high-level policy down the hierarchy, so that the low-level policy can reason over every plausible sub-goal at once. The high-level policy π_{hi} is the point-cloud policy of Appendix I of the supplementary material. It predicts a per-anchor Gaussian mixture over the next sub-goal, in which every scene point proposes a gripper pose relative to itself and votes on its plausibility, and it is trained with transition-focused sampling and label swapping so that the mixture places mass on both the outgoing and the incoming sub-goal near a boundary instead of hedging between them. At an ambiguous transition the uncertainty therefore appears directly in the output as several modes of comparable weight.

The low-level policy π_{lo} shares GIST’s backbone, but the auxiliary goal head is removed and the J highest-weight candidates from π_{hi} enter as inputs through two routes (Details in Appendix I of the supplementary material). Each goal candidate is encoded into a token appended to the RoPE trunk, anchored in 3D at its end-effector keypoint and in time at the end of the action horizon, so that the visual tokens can relate themselves to it at true metric offset. A separate encoding of the same candidates is consumed by weighted cross-attention inside the DiT. In both routes the mixture weights bias the attention logits towards the likelier candidates. During training π_{lo} consumes the mixtures that π_{hi} predicts on the demonstration frames, occasionally replaced by a sparse ground-truth mixture, and full details are given in Appendix I of the supplementary material. Since the backbone is shared, the only difference from GIST is that the sub-goal enters as an input distribution rather than as an auxiliary training target, which is what Q2 asks.

D. *Implementation Details*

We train one policy per task for every method, with no language conditioning or task identifier, and all low-level policies including the baselines share the same architecture and training settings. Each observes the two most recent frames, predicts $H=16$ actions and executes the first $H_{exec}=8$ before replanning. Every model is trained from scratch on one H100, for 100 epochs on the MimicGen tasks (51.5k steps) and 200 on Push-T (37.2k steps). We set $c_1=0.1$, at which the auxiliary gradient at the shared encoder is about a quarter of the flow-matching gradient at initialisation, and

the no-auxiliary-loss baseline simply omits the goal head. The hierarchical baselines keep the $J=6$ highest-weight candidates from π_{hi} and run the two levels synchronously, querying π_{hi} once per observation while π_{lo} executes its H_{exec} actions, so each plan reacts to the current scene. All remaining hyperparameters, our hardware setup and per-task data collection details are given in Appendices III, IV and V of the supplementary material.

E. *Results*

Does auxiliary sub-goal supervision improve performance over the baselines? Table I answers Q1 and Q2. GIST achieves the highest success rate on three of the four simulation tasks and the highest average at 60.2%. Removing the auxiliary loss while holding everything else fixed drops the average to 49.5%, a loss of **10.7%**, so the supervision itself rather than the architecture that carries it accounts for the gain, and replacing the mixture head with single sub-goal regression recovers only part of it at 54.7%, still **5.5%** behind (Q1). The gap to the regression head is largest on Coffee Preparation, 68.0% against 50.0%, where the sub-task transitions are most ambiguous and a regressed target averages the present and the next sub-goal into a pose that matches neither. The standard hierarchical policy reaches 45.0%, **15.2%** below GIST and worse than the policy trained with no sub-goal information at all (Q2), since a high-level model that commits to a single sub-goal is often wrong near a boundary and the low-level policy cannot recover. Our uncertainty-aware hierarchical baseline, which passes the full mixture to the low-level policy, closes much of that gap at 54.3%, yet remains **5.9%** below GIST while running a high-level model at inference (Q2). The bottleneck is therefore not how the low-level policy consumes the sub-goal but that the sub-goal enters as an input at all.

Does the gain transfer to the real world? Table II reports full and partial task success on the three real-world non-prehensile tasks. GIST outperforms the policy trained without the auxiliary loss by **33.3%** in full task success averaged over the three tasks, and its higher partial task success indicates that, when it does fail, it fails later in the task. The characteristic failure of the policy without the auxiliary loss is a lack of guidance at points where the task admits more than one valid solution: without a supervised signal for which sub-goal is in effect, the policy has no basis for committing to one of several equally plausible paths, stalling the progress of the rollout.

How much does the choice of sub-goal extraction matter? Table III trains π_{aux} with each of the six extraction procedures, so the placement of the sub-goal boundaries is the only thing that varies. The five procedures evaluated on all four tasks average between 57.4% and 60.2%, a spread of 2.8 points, and every procedure stays well clear of the policy trained without sub-goal supervision at 49.5%. No procedure dominates. AWE greedy, the setting used in Table I, is the strongest overall yet the weakest on Hammer Cleanup, where the fixed-interval control that places a boundary every $K=20$ is the best. Combined with the results above, this indicates

TABLE III

EFFECT OF THE SUB-GOAL EXTRACTION PROCEDURE ON π_{aux} . EVERY ROW TRAINS THE SAME POLICY WITH THE SAME SCHEDULE AND DIFFERS ONLY IN WHERE THE SUB-GOAL BOUNDARIES ARE PLACED. COLUMNS ARE AS IN TABLE I. BEST PER COLUMN IN BOLD.

Extraction procedure	Kitchen D1	Coffee Prep. D1	Hammer Cleanup D1	MimicGen Avg.	Push-T	Avg.
Mixed gripper and orientation heuristic	46.0	58.7	57.3	54.0	N.A.	54.0
Fixed interval, $K=20$	58.0	52.0	59.3	56.4	61.3	57.7
Qwen3.6	56.2	60.7	54.0	57.0	58.7	57.4
Bayesian velocity changepoints	62.7	62.0	54.0	59.6	56.0	58.7
AWE greedy, $th = 0.35$	60.7	68.0	50.0	59.6	62.0	60.2
AWE dynamic programming, $th = 0.05$	65.3	56.7	58.0	60.0	58.0	59.5

TABLE IV

COMPARISON WITH DIFFUSION POLICY ON FIVE MIMICGEN TASKS, ALL TRAINED ON 200 DEMONSTRATIONS FOR 250 EPOCHS. WE REPORT SUCCESS RATE (%). DP-C AND DP-T ARE THE CONVOLUTIONAL AND TRANSFORMER VARIANTS OF DIFFUSION POLICY [1], WITH RELATIVE OR ABSOLUTE ACTION SPACES. PUSH-T IS SCORED BY TARGET AREA COVERAGE, THE METRIC USED BY DIFFUSION POLICY, WHICH IS THE CLIPPED RATIO OF COVERED AREA TO THE 0.95 SUCCESS THRESHOLD, MAXIMISED OVER THE EPISODE AND AVERAGED. IT IS REPORTED SEPARATELY AND IS NOT INCLUDED IN THE AVERAGE. BEST PER COLUMN IN BOLD.

Method	Success rate (%)						Coverage (%)
	Hammer Cleanup D1	Kitchen D1	Coffee Prep. D1	Mug Cleanup D1	Square D2	MimicGen Avg.	Push-T
DP-C, relative	54.0	64.0	53.0	39.0	13.0	44.6	–
DP-T, absolute	60.0	75.0	51.0	43.0	11.0	48.0	–
DP-C, absolute	59.0	85.0	62.0	59.0	19.0	56.8	91.0
No auxiliary loss	58.3	64.7	61.3	50.0	32.7	53.4	–
GIST (ours)	64.0	69.7	75.3	56.7	43.3	61.8	95.4

that what matters is the presence of spatial goal supervision rather than its precise definition, and GIST does not depend on a carefully engineered subgoal extractor (Q3).

Comparison with Diffusion Policy on MimicGen tasks. Table IV compares GIST with three Diffusion Policy variants across five MimicGen tasks spanning prehensile and non-prehensile manipulation, all trained on 200 demonstrations for 250 epochs. GIST attains the highest MimicGen average, beating the strongest variant by **5.0%**, and leads on three of the five tasks. It also leads on Push-T, at 95.4% target area coverage against 91.0%. The margin is widest on Square D2, at 43.3% against 19.0%.

VI. CONCLUSION AND FUTURE WORK

We present GIST, an end-to-end visuomotor policy that uses sub-goals purely as a training signal, rather than through an explicit hierarchy in which a high-level model predicts the sub-goals and a low-level policy is conditioned on them. The auxiliary head predicts a Gaussian mixture over the next sub-goal gripper pose during training and is discarded during rollout. This becomes particularly important in tasks with a non-prehensile component, where the gripper holds a fixed configuration and the discrete open-close signal that hierarchical policies depend on, both to extract their sub-goals and to learn from them, is unavailable. Across four simulation and three real-world tasks, GIST outperforms a standard sub-goal based hierarchical policy by **15.2%** and a stronger uncertainty-aware hierarchical baseline that we formulate by **5.9%**. Removing the auxiliary head costs **10.7%** in simulation and **33.3%** in the real world, and regressing a single sub-goal instead of a mixture costs **5.5%**, so both the

supervision and its representation as a distribution matter. Performance barely changes across six subgoal extraction procedures, indicating that our method is robust to the choice of the subgoal decomposition method.

REFERENCES

- [1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2024.
- [5] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.
- [6] A. Goyal, V. Blukis, J. Xu, Y. Guo, Y.-W. Chao, and D. Fox, “RVT-2: Learning precise manipulation from few demonstrations,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [7] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki, “3D diffuser actor: Policy diffusion with 3D scene representations,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2024.
- [8] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani, “Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.
- [9] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [10] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar, “MimicPlay: Long-horizon imitation learning by watching human play,” in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.

- [11] Y. Wang, Z. Wang, M. Nakura, P. Bhowal, C.-L. Kuo, Y.-T. Chen, Z. Erickson, and D. Held, "ArticBot: Learning universal articulated object manipulation policy via large scale simulation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [12] S. Krishna, B. Eisner, H. Zhan, Y. Yuan, H. Zhen, C. Gan, S. Tulsiani, and D. Held, "GHOST: Hierarchical sub-goal policies for generalizing robot manipulation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2026.
- [13] Y. Li, Y. Deng, J. Zhang, J. Jang, M. Memmel *et al.*, "HAMSTER: Hierarchical action models for open-world robot manipulation," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [14] M. T. Mason, "Mechanics and planning of manipulator pushing operations," *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 53–71, 1986.
- [15] K. M. Lynch and M. T. Mason, "Stable pushing: Mechanics, controllability, and planning," *The International Journal of Robotics Research*, vol. 15, no. 6, pp. 533–556, 1996.
- [16] S. Goyal, A. Ruina, and J. Papadopoulos, "Planar sliding with dry friction. Part 1. Limit surface and moment function," *Wear*, vol. 143, no. 2, pp. 307–330, 1991.
- [17] K. M. Lynch and M. T. Mason, "Dynamic nonprehensile manipulation: Controllability, planning, and experiments," *The International Journal of Robotics Research*, vol. 18, no. 1, pp. 64–92, 1999.
- [18] F. R. Hogan and A. Rodriguez, "Reactive planar non-prehensile manipulation with hybrid model predictive control," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 755–773, 2020.
- [19] J. Moura, T. Stouraitis, and S. Vijayakumar, "Non-prehensile planar manipulation via trajectory optimization with complementarity constraints," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 970–976.
- [20] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, "Contact mode guided motion planning for quasidynamic dexterous manipulation in 3D," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2730–2736.
- [21] T. Pang, H. J. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Transactions on Robotics*, vol. 39, no. 6, pp. 4691–4711, 2023.
- [22] H. J. T. Suh, T. Pang, T. Zhao, and R. Tedrake, "Dexterous contact-rich manipulation via the contact trust region," *arXiv preprint arXiv:2505.02291*, 2025.
- [23] W. Zhou, B. Jiang, F. Yang, C. Paxton, and D. Held, "HACMan: Learning hybrid actor-critic maps for 6D non-prehensile manipulation," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [24] B. Jiang, Y. Wu, W. Zhou, C. Paxton, and D. Held, "HACMan++: Spatially-grounded motion primitives for manipulation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [25] W. Zhou and D. Held, "Learning to grasp the ungraspable with emergent extrinsic dexterity," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.
- [26] Y. Cho, J. Han, Y. Cho, and B. Kim, "CORN: Contact-based object representation for non-prehensile manipulation of general unseen objects," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [27] Y. Cho, J. Han, J. Han, and B. Kim, "Hierarchical and modular network on non-prehensile manipulation in general environments," *arXiv preprint arXiv:2502.20843*, 2025.
- [28] H. Jung, D. Lee, H. Park, J. Kim, and B. Kim, "SPIN: Distilling Skill-RRT for long-horizon prehensile and non-prehensile manipulation," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2025.
- [29] P. Agrawal, A. Nair, P. Abbeel, J. Malik, and S. Levine, "Learning to poke by poking: Experiential learning of intuitive physics," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [30] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 2786–2793.
- [31] W. Li, H. Zhao, Z. Yu, Y. Du, Q. Zou, R. Hu, and K. Xu, "PIN-WM: Learning physics-informed world models for non-prehensile manipulation," in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [32] J. Lyu, Z. Li, X. Shi, C. Xu, Y. Wang, and H. Wang, "DyWA: Dynamics-adaptive world action model for generalizable non-prehensile manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [33] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2021.
- [34] H. Zhou and K. Jia, "BiNoMaP: Learning category-level bi-manual non-prehensile manipulation primitives," *arXiv preprint arXiv:2509.21256*, 2025.
- [35] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [36] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis *et al.*, "RT-1: Robotics transformer for real-world control at scale," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [37] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black *et al.*, "Octo: An open-source generalist robot policy," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [38] NVIDIA, "GR00T N1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.
- [39] X. Li, V. Belagali, J. Shang, and M. S. Ryoo, "Crossway diffusion: Improving diffusion-based visuomotor policy via self-supervised learning," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [40] R. Zheng *et al.*, "FLARE: Robot learning with implicit world modeling," *arXiv preprint arXiv:2505.15659*, 2025.
- [41] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [42] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [43] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec *et al.*, "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research (TMLR)*, 2024.
- [44] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.
- [45] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "MimicGen: A data generation system for scalable robot learning using human demonstrations," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [46] L. X. Shi, A. Sharma, T. Z. Zhao, and C. Finn, "Waypoint-based imitation learning for robotic manipulation," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [47] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.